ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 9

1. Աշխատանքի նպատակը

Աշխատանքի նպատակն է ուսումնասիրել C++ ծրագրավորման լեզվում դինամիկ հիշողության կառավարման սկզբունքները, ստեղծել դաս՝ դինամիկ զանգվածների օգտագործմամբ, իրականացնել պատճենող կոնստրուկտոր, վերագրման օպերատոր և դեստրուկտոր՝ պահպանելով Rule of Three սկզբունքը:

Նաև նպատակ է կիրառել Էյլերի թվային մեթոդը առաջին կարգի դիֆերենցիալ հավասարման Կոշիի խնդիրը լուծելու, ստացված արդյունքները աղյուսակով ներկայացնելու և առավելագույն արժեքը որոշելու համար:

2. Խնդրի պահանջը

Տրված է Կոշիի հետևյալ խնդիրը՝

y′=ln⁡(x+1)+e−x−0.3y*y*′=ln(*x*+1)+*e*−*x*−0.3*y*

Սկզբնական պայմանը՝

y(0)=0.5*y*(0)=0.5

Անհրաժեշտ է C++ լեզվով ստեղծել առանձին CauchyProblem դաս, որի private դաշտերում նախատեսված կլինեն double* տիպի դինամիկ զանգվածներ և դրանց չափը:

Ծրագիրը պետք է իրականացնի հետևյալ գործողությունները.

1. Ստեղծել պարամետրերով կոնստրուկտոր:
2. Իրականացնել պատճենող կոնստրուկտոր՝ խորքային պատճենմամբ:
3. Իրականացնել վերագրման օպերատոր և դեստրուկտոր:
4. Էյլերի մեթոդով հաշվել լուծման 21 արժեք՝ ընդունելով h=0.05*h*=0.05, N=20*N*=20:
5. Ստացված արդյունքները պահել դինամիկ զանգվածներում:
6. Արդյունքները ներկայացնել աղյուսակով՝ տպելով յուրաքանչյուր հինգերորդ կետի արժեքը:
7. Գտնել y*y*-ի առավելագույն արժեքը:
8. Ստուգել երկու անկախ օբյեկտների և դրանց խորքային պատճենման ճիշտ աշխատանքը:

9. Մուտքային սխալ արժեքների դեպքում արտածել հասկանալի հաղորդագրություն:

Լոգարիթմի սահմանման տիրույթի պայմանն է՝

$x+1>0 \Rightarrow x>-1 x+1>0 \Rightarrow x>-1$

3. Առաջադրանքի լուծումը

cpp

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <stdexcept>
using namespace std;

class CauchyProblem {
private:
   double* x;
   double* y;
   int size;
   double h;

public:
   // Պարամետրերով կոնստրուկտոր
   CauchyProblem(int n, double step, double y0)
      : x(nullptr), y(nullptr), size(n), h(step) {

      if (n < 2 || step <= 0 || !isfinite(step)
         || !isfinite(y0)) {
         throw invalid_argument("Invalid input parameters!");
```

```
    }

    x = new double[size];

    try {
        y = new double[size];
    } catch (...) {
        delete[] x;
        throw;
    }

    for (int i = 0; i < size; i++) {
        x[i] = 0.0;
        y[i] = 0.0;
    }

    x[0] = 0.0;
    y[0] = y0;
}

// Պատճենող կոնստրուկտոր՝ խորքային պատճենում
CauchyProblem(const CauchyProblem& other)
    : x(nullptr), y(nullptr),
      size(other.size), h(other.h) {

    x = new double[size];
```

```
    try {
        y = new double[size];
    } catch (...) {
        delete[] x;
        throw;
    }

    for (int i = 0; i < size; i++) {
        x[i] = other.x[i];
        y[i] = other.y[i];
    }
}

// Վերագրման օպերատոր
CauchyProblem& operator=(const CauchyProblem& other) {
    if (this == &other) {
        return *this;
    }

    double* newX = new double[other.size];
    double* newY = nullptr;

    try {
        newY = new double[other.size];
    } catch (...) {
        delete[] newX;
        throw;
```

```
    }

    for (int i = 0; i < other.size; i++) {
        newX[i] = other.x[i];
        newY[i] = other.y[i];
    }

    delete[] x;
    delete[] y;

    x = newX;
    y = newY;
    size = other.size;
    h = other.h;

    return *this;
}

// Դեստրուկտոր
~CauchyProblem() {
    delete[] x;
    delete[] y;
}

// Էյլերի մեթոդով հաշվարկ
void calculate() {
    for (int i = 0; i < size - 1; i++) {
```

```
        double f = log(x[i] + 1.0)
                + exp(-x[i])
                - 0.3 * y[i];

        x[i + 1] = x[i] + h;
        y[i + 1] = y[i] + h * f;
    }
}

// Արդյունքների արտածում
void printTable() const {
    cout << fixed << setprecision(6);
    cout << "\n i\t   x[i]\t\t   y[i]\n";
    cout << "-----------------------------------\n";

    for (int i = 0; i < size; i += 5) {
        cout << i << "\t"
             << x[i] << "\t\t"
             << y[i] << '\n';
    }

    if ((size - 1) % 5 != 0) {
        int i = size - 1;
        cout << i << "\t"
             << x[i] << "\t\t"
             << y[i] << '\n';
    }
```

```
}

// Առավելագույն y արժեքի որոշում
double maxY() const {
    double maximum = y[0];

    for (int i = 1; i < size; i++) {
        if (y[i] > maximum) {
            maximum = y[i];
        }
    }

    return maximum;
}

// Արդյունքների համեմատություն
bool sameResults(const CauchyProblem& other) const {
    if (size != other.size || h != other.h) {
        return false;
    }

    for (int i = 0; i < size; i++) {
        if (x[i] != other.x[i] || y[i] != other.y[i]) {
            return false;
        }
    }
```

```
        return true;
    }

    // Սկզբնական արժեքի փոփոխություն
    void setInitialY(double value) {
        if (!isfinite(value)) {
            throw invalid_argument("Invalid initial value!");
        }

        y[0] = value;
    }
};

int main() {
    try {
        const int N = 20;
        const double h = 0.05;
        const double y0 = 0.5;

        // Առաջին օբյեկտ
        CauchyProblem first(N + 1, h, y0);
        first.calculate();

        cout << "Original object:";
        first.printTable();

        cout << "\nMaximum y = "
```

```
        << fixed << setprecision(6)
        << first.maxY() << '\n';

    // Պատճենող կոնստրուկտորի ստուգում
    CauchyProblem second(first);

    cout << "\nCopy constructor test: "
        << (first.sameResults(second)
            ? "PASSED" : "FAILED") << '\n';

    // Վերագրման օպերատորի ստուգում
    CauchyProblem third(N + 1, h, 0.0);
    third = first;

    cout << "Assignment operator test: "
        << (first.sameResults(third)
            ? "PASSED" : "FAILED") << '\n';

    // Խորքային պատճենման ստուգում
    second.setInitialY(0.6);
    second.calculate();

    cout << "Original and copy are independent: "
        << (first.sameResults(second)
            ? "NO" : "YES") << '\n';
}
catch (const exception& e) {
```

```
        cerr << "Error: " << e.what() << '\n';
        return 1;
    }

    return 0;
}
```